# FocusOn

## Plan de corrección técnica

| | |
|---|---|
| **Versión** | v1.0 |
| **Fecha** | Abril 2026 |
| **Proyecto** | FocusOn — app de recordatorios |
| **Estado** | 4 errores críticos identificados |

# Introducción

Este documento recoge en detalle los cuatro errores críticos y las dos deudas técnicas identificadas en el análisis de la documentación de FocusOn. Para cada problema se explica qué está fallando actualmente, por qué es importante corregirlo y exactamente qué cambios hay que hacer en qué ficheros, incluyendo el código completo listo para implementar.

Los problemas están ordenados por prioridad: los dos primeros afectan al núcleo del producto (las notificaciones) y deben resolverse antes de pasar a los siguientes.

## Resumen del estado actual

| Área | Estado actual | Estado objetivo |
|---|---|---|
| **Notificaciones repetitivas** | Campo en DB, sin implementar | Funcional y cancelable |
| **Toggle notificaciones** | Roto al reactivar | Reactiva correctamente |
| **Racha de días** | Se rompe con resetDay() | Persistida en daily_log |
| **Historial / stats** | Se pierde al borrar tareas | Tabla daily_log permanente |
| **DayPicker duplicado** | Código duplicado x2 | Componente compartido |
| **Frases motivacionales** | Misma frase cada lunes | Una por día del año |

# Error 1 — Notificaciones repetitivas no implementadas

## El problema

El campo repeat_minutes existe en la tabla tasks y aparece en el formulario de creación de tareas con opciones de 30 min, 1h, 2h y 3h. Sin embargo, la función scheduleTaskNotification en lib/notifications.ts usa exclusivamente un trigger de tipo DATE, que dispara la notificación una única vez en la fecha y hora indicadas. Una vez disparada, si el usuario no la ve o no completa la tarea, no vuelve a aparecer.

El objetivo declarado de la app es precisamente el opuesto: los recordatorios deben repetirse hasta que la tarea se complete. Tal como está, el núcleo del producto no funciona.

**Impacto**

Un usuario que no vea la notificación inicial (pantalla bloqueada, modo no molestar, distracción) no recibe ningún aviso adicional. La tarea queda olvidada. Esto contradice directamente el propósito de la app.

## La solución

Hay que separar la lógica en dos notificaciones para cada tarea que tenga repeat_minutes > 0: una notificación principal a la hora programada, y una segunda notificación de seguimiento con un trigger de intervalo en segundos que se repita indefinidamente. Al marcar la tarea como completada, ambas notificaciones deben cancelarse usando sus IDs respectivos. Para ello hay que guardar los dos IDs en la base de datos.

### Paso 1 — Añadir columna repeat_notification_id en la base de datos

**Fichero:**

Añade una nueva columna para guardar el ID de la notificación de repetición de forma independiente al ID de la notificación principal. Esto permite cancelar cada una por separado.

```
// En la función initDB(), dentro del CREATE TABLE:
notification_id TEXT,              -- notificación principal (ya existe)
repeat_notification_id TEXT        -- notificación de repetición (nueva)

// Añade también la migración para bases de datos ya existentes:
db.runSync(`
  ALTER TABLE tasks
  ADD COLUMN repeat_notification_id TEXT
`);
// Envuelve el ALTER en try/catch por si la columna ya existe:
try {
  db.runSync("ALTER TABLE tasks ADD COLUMN repeat_notification_id TEXT");
} catch (_) {
  // La columna ya existe, ignorar
}
```

### Paso 2 — Actualizar el tipo Task

**Fichero:** y

```
export interface Task {
  // ... campos existentes ...
  notification_id: string | null;
  repeat_notification_id: string | null;  // añadir esta línea
}
```

### Paso 3 — Reescribir scheduleTaskNotification

**Fichero:**

La función debe ahora devolver un objeto con dos IDs en lugar de uno solo. Si repeat_minutes es 0 o nulo, repeat_notification_id será null.

```
export async function scheduleTaskNotification(task: Task): Promise<{
  notification_id: string | null;
  repeat_notification_id: string | null;
}> {
  if (!task.reminder_time || !task.scheduled_date) {
```

```
    return { notification_id: null, repeat_notification_id: null };
  }

  const [h, m] = task.reminder_time.split(":").map(Number);
  const triggerDate = new Date(task.scheduled_date);
  triggerDate.setHours(h, m, 0, 0);

  if (triggerDate <= new Date()) {
    return { notification_id: null, repeat_notification_id: null };
  }

  // Notificación principal (única, en la hora exacta)
  const notification_id = await Notifications.scheduleNotificationAsync({
    content: {
      title: "Tarea pendiente",
      body: getMotivationalMessage(task.title),
      data: { taskId: task.id },
      sound: true,
    },
    trigger: triggerDate,
  });

  // Notificación de repetición (solo si repeat_minutes > 0)
  let repeat_notification_id: string | null = null;
  if (task.repeat_minutes && task.repeat_minutes > 0) {
    repeat_notification_id = await Notifications.scheduleNotificationAsync({
      content: {
        title: "¿Ya lo hiciste?",
        body: `${task.title} sigue pendiente`,
        data: { taskId: task.id },
        sound: true,
      },
      trigger: {
        seconds: task.repeat_minutes * 60,
        repeats: true,
      },
    });
  }

  return { notification_id, repeat_notification_id };
}
```

## Paso 4 — Actualizar cancelNotification

**Fichero:**

```
export async function cancelTaskNotifications(
  notification_id: string | null,
  repeat_notification_id: string | null
): Promise<void> {
  if (notification_id) {
    await Notifications.cancelScheduledNotificationAsync(notification_id);
  }
  if (repeat_notification_id) {
    await
Notifications.cancelScheduledNotificationAsync(repeat_notification_id);
  }
}
```

## Paso 5 — Actualizar el Zustand store

**Fichero:**

Todos los métodos que llaman a scheduleTaskNotification o cancelNotification deben actualizarse para manejar los dos IDs. Los cambios afectan a addTask, editTask, removeTask y toggleTask.

```
  // addTask: guardar ambos IDs
  addTask: async (newTask) => {
    const id = await createTask(newTask);
    const task = await getTaskById(id);
    const { notification_id, repeat_notification_id } =
      await scheduleTaskNotification(task);
    await updateTask(id, { notification_id, repeat_notification_id });
    // ... recargar tareas
  },

  // toggleTask: cancelar ambas al completar, reprogramar al revertir
  toggleTask: async (id) => {
    const task = await getTaskById(id);
    const completing = !task.is_completed;
    await toggleTaskComplete(id, completing);
    if (completing) {
      await cancelTaskNotifications(
        task.notification_id,
        task.repeat_notification_id
      );
      await updateTask(id, {
        notification_id: null,
        repeat_notification_id: null
      });
    } else {
      const updated = await getTaskById(id);
      const ids = await scheduleTaskNotification(updated);
      await updateTask(id, ids);
    }
    // ... recargar tareas
  },
```

# Error 2 — Toggle de notificaciones roto al reactivar

## El problema

En la pantalla de Settings hay un toggle para activar o desactivar las notificaciones. La documentación reconoce explícitamente el fallo: al desactivar se cancelan todas las notificaciones existentes, pero al volver a activar no se reprograman. El usuario queda sin recordatorios de forma silenciosa, sin ningún mensaje de advertencia.

**Por qué es crítico**

Un usuario que accidentalmente desactive el toggle y lo vuelva a activar creerá que sus notificaciones están funcionando cuando en realidad se han perdido todas. Es un fallo silencioso, el peor tipo.

## La solución

**Fichero:** y

Cuando el usuario reactive el toggle, hay que recorrer todas las tareas pendientes que tienen reminder_time y scheduled_date en el futuro, reprogramar sus notificaciones y actualizar los IDs en la base de datos.

### Paso 1 — Añadir método rescheduleAll al store

```
// store/useTaskStore.ts
rescheduleAll: async () => {
  const allTasks = await getAllTasks();
  const pending = allTasks.filter(t => !t.is_completed && t.reminder_time);
  for (const task of pending) {
    const ids = await scheduleTaskNotification(task);
    await updateTask(task.id, ids);
  }
  await get().loadTasks();
},
```

### Paso 2 — Actualizar el toggle en Settings

```
// app/(tabs)/settings.tsx
const { rescheduleAll } = useTaskStore();

const handleNotificationsToggle = async (enabled: boolean) => {
  setNotificationsEnabled(enabled);
  if (!enabled) {
    await cancelAllNotifications();
  } else {
    // Reprogramar todas las notificaciones pendientes
    await rescheduleAll();
    Alert.alert(
      "Notificaciones activadas",
      "Se han reprogramado todos tus recordatorios."
    );
  }
};

// En el JSX, reemplazar la llamada actual:
<Switch
  value={notificationsEnabled}
  onValueChange={handleNotificationsToggle}  // <- antes solo cancelaba
/>
```

# Error 3 — La racha se rompe al usar resetDay()

## El problema

La función calcStreak() calcula la racha buscando días consecutivos en el campo completed_at de las tareas. Cuando el usuario llama a resetDay(), la función toggleTaskComplete se invoca con completed = false para todas las tareas, borrando el

valor de completed_at. Resultado: el historial de ese día desaparece completamente y la racha se rompe, aunque el usuario sí había completado tareas.

El mismo problema ocurre si el usuario borra tareas completadas manualmente: el historial se pierde. Para una app basada en la motivación por rachas, esto es especialmente dañino.

## La solución — tabla daily_log

La racha no debe depender de los datos en tiempo real de la tabla tasks. Necesita una tabla de solo escritura que registre los logros del día de forma permanente, independiente de resets o borrados.

### Paso 1 — Crear la tabla daily_log

**Fichero:**

```
// Añadir en initDB(), después de CREATE TABLE tasks:
db.runSync(`
  CREATE TABLE IF NOT EXISTS daily_log (
    date TEXT PRIMARY KEY,        -- YYYY-MM-DD
    completed_count INTEGER DEFAULT 0,
    total_count INTEGER DEFAULT 0
  )
`);
```

### Paso 2 — Función para actualizar el log del día

```
export function updateDailyLog(date: string): void {
  const rows = db.getAllSync<{ total: number; completed: number }>(
    `SELECT
      COUNT(*) as total,
      SUM(CASE WHEN is_completed = 1 THEN 1 ELSE 0 END) as completed
     FROM tasks
     WHERE scheduled_date = ?`,
    [date]
  );
  const { total, completed } = rows[0] ?? { total: 0, completed: 0 };
  db.runSync(
    `INSERT INTO daily_log (date, completed_count, total_count)
     VALUES (?, ?, ?)
     ON CONFLICT(date) DO UPDATE SET
       completed_count = excluded.completed_count,
       total_count = excluded.total_count`,
    [date, completed, total]
  );
}

export function getDailyLog(): { date: string; completed_count: number;
total_count: number }[] {
  return db.getAllSync(`SELECT * FROM daily_log ORDER BY date DESC`);
}
```

### Paso 3 — Llamar a updateDailyLog en los momentos correctos

**Fichero:**

Hay que actualizar el log cada vez que el estado de una tarea cambia o el día se reinicia. Son cuatro sitios:

```
// 1. En toggleTask, después de toggleTaskComplete:
await toggleTaskComplete(id, completing);
await updateDailyLog(task.scheduled_date);  // <- añadir

// 2. En addTask, después de crear la tarea:
await updateDailyLog(newTask.scheduled_date);  // <- añadir

// 3. En removeTask, después de eliminar:
await updateDailyLog(task.scheduled_date);  // <- añadir

// 4. En resetDay, después de desmarcar todas las tareas:
const today = new Date().toISOString().split("T")[0];
// Nota: resetDay NO sobreescribe el log del día anterior
// Solo actualiza el día actual con los nuevos valores
await updateDailyLog(today);  // <- añadir
```

## Paso 4 — Reescribir calcStreak usando daily_log

**Fichero:**

La lógica de la racha ahora lee de daily_log en lugar de buscar en completed_at de tasks. Un día cuenta para la racha si completed_count > 0.

```
function calcStreak(log: DailyLogEntry[]): number {
  let streak = 0;
  const today = new Date();
  for (let i = 0; i < 365; i++) {
    const d = new Date(today);
    d.setDate(d.getDate() - i);
    const iso = d.toISOString().split("T")[0];
    const entry = log.find(l => l.date === iso);
    if (entry && entry.completed_count > 0) {
      streak++;
    } else if (i > 0) {
      // Permitir que el día de hoy no tenga tareas aún
      break;
    }
  }
  return streak;
}
```

## Paso 5 — Actualizar el gráfico semanal

El gráfico de barras de Stats actualmente cuenta tareas por completed_at. Ahora puede leer directamente de daily_log, que es más eficiente y no se ve afectado por borrados:

```
// En stats.tsx, reemplazar la lógica de getLast7Days:
const last7 = useMemo(() => {
  return Array.from({ length: 7 }, (_, i) => {
    const d = new Date();
    d.setDate(d.getDate() - (6 - i));
    const iso = d.toISOString().split("T")[0];
```

```
      const entry = dailyLog.find(l => l.date === iso);
      return {
        date: iso,
        completed: entry?.completed_count ?? 0,
        total: entry?.total_count ?? 0,
      };
    });
  }, [dailyLog]);
```

# Error 4 — Historial de estadísticas se pierde al borrar

## El problema

Los porcentajes de completado y el conteo total en la pantalla de Stats se calculan sobre el array tasks en memoria, que se carga con getAllTasks() o getTasksByDate(). Si el usuario usa la opción "Eliminar todo" de Settings, o borra tareas completadas para limpiar la lista, todos esos datos desaparecen y las estadísticas vuelven a cero.

> **Consecuencia**
> Completar un ciclo de 30 días y luego borrar las tareas antiguas significa perder toda evidencia del progreso. Para un usuario motivado por los datos y las rachas, esto puede ser desmotivador.

## La solución

Este error queda resuelto como efecto secundario de la corrección del Error 3. La tabla daily_log actúa como historial permanente porque:

- updateDailyLog hace un INSERT ... ON CONFLICT DO UPDATE, nunca borra filas.
- purgeAll() en el store borra tasks pero no debe borrar daily_log.
- Las estadísticas históricas leen de daily_log, no de tasks.

### Cambio necesario en purgeAll()

**Fichero:** y

```
// lib/db.ts — deleteAllTasks NO debe tocar daily_log
// Verificar que la función solo borra la tabla tasks:
export function deleteAllTasks(): void {
  db.runSync("DELETE FROM tasks");
  // NO añadir DELETE FROM daily_log aquí
}

// Si quieres dar al usuario la opción de borrar también el historial,
// añade una función separada y explícita:
export function deleteHistory(): void {
  db.runSync("DELETE FROM daily_log");
}
```

### Actualizar Stats para leer de daily_log

**Fichero:**

```
// En el useFocusEffect de stats.tsx:
useFocusEffect(useCallback(() => {
  loadTasks();           // para stats del día actual
  loadDailyLog();        // para historial persistido
}, []));

// Estadísticas históricas desde daily_log:
const historicStats = useMemo(() => {
  const totalCompleted = dailyLog.reduce((acc, d) => acc + d.completed_count,
0);
  const totalCreated = dailyLog.reduce((acc, d) => acc + d.total_count, 0);
  const pct = totalCreated > 0
    ? Math.round((totalCompleted / totalCreated) * 100)
    : 0;
  return { totalCompleted, totalCreated, pct };
}, [dailyLog]);
```

# Deuda técnica 1 — DayPicker duplicado

## El problema

El componente DayPicker está definido localmente dentro de  y de . La única diferencia entre ambas versiones es el número de días que muestran (7 y 14 respectivamente). Cualquier corrección de bug o mejora visual hay que aplicarla dos veces, y es fácil que las versiones diverjan.

## La solución

### Crear components/DayPicker.tsx

```
// components/DayPicker.tsx
import React from "react";
import { ScrollView, TouchableOpacity, Text, StyleSheet, View } from "react-
native";
import { format, addDays, isToday, isTomorrow } from "date-fns";
import { es } from "date-fns/locale";
import { Colors, Spacing, FontSize, Radius } from "@/constants/theme";

interface DayOption {
  iso: string;
  label: string;
  sublabel: string;
}

function buildDays(count: number): DayOption[] {
  return Array.from({ length: count }, (_, i) => {
    const date = addDays(new Date(), i);
    const iso = format(date, "yyyy-MM-dd");
    const label = isToday(date)
```

```
      ? "Hoy"
      : isTomorrow(date)
      ? "Mañana"
      : format(date, "EEE", { locale: es });
    const sublabel = format(date, "d MMM", { locale: es });
    return { iso, label, sublabel };
  });
}

interface DayPickerProps {
  selected: string;
  onChange: (iso: string) => void;
  daysAhead?: number;  // default: 7
}

export function DayPicker({ selected, onChange, daysAhead = 7 }: DayPickerProps)
{
  const days = buildDays(daysAhead);
  return (
    <ScrollView horizontal showsHorizontalScrollIndicator={false}>
      {days.map(day => (
        <TouchableOpacity
          key={day.iso}
          onPress={() => onChange(day.iso)}
          style={[styles.chip, selected === day.iso && styles.chipActive]}
        >
          <Text style={[styles.label, selected === day.iso &&
styles.labelActive]}>
            {day.label}
          </Text>
          <Text style={[styles.sub, selected === day.iso && styles.subActive]}>
            {day.sublabel}
          </Text>
        </TouchableOpacity>
      ))}
    </ScrollView>
  );
}
```

### Actualizar los modales para usar el componente compartido

```
// app/task/new.tsx — reemplazar el DayPicker local por:
import { DayPicker } from "@/components/DayPicker";
// ...
<DayPicker selected={selectedDate} onChange={setSelectedDate} daysAhead={7} />

// app/task/[id].tsx — reemplazar el DayPicker local por:
import { DayPicker } from "@/components/DayPicker";
// ...
<DayPicker selected={selectedDate} onChange={setSelectedDate} daysAhead={14} />
```

# Deuda técnica 2 — Frases motivacionales siempre iguales

## El problema

El mensaje del día en Stats se elige con getDay(), que devuelve el día de la semana (0-6). Esto significa que todos los lunes se muestra la misma frase, todos los martes la misma, y así sucesivamente. Con solo 7 frases posibles, el usuario las memoriza rápidamente y dejan de tener impacto motivacional.

### La solución

Usar el día del año (0-364) como semilla para seleccionar una frase de un array más amplio. Con esta aproximación se pueden tener 30-50 frases y el usuario verá una diferente cada día durante semanas antes de que se repita.

```
// constants/messages.ts — ampliar el array a 30+ frases
export const DAILY_MESSAGES = [
  "El Miguel de mañana te lo agradecerá.",
  "Una tarea a la vez. Eso es todo.",
  "No lo dejes para mañana lo que puedes terminar hoy.",
  "Cada tarea completada es un paso adelante.",
  "La constancia vence al talento.",
  // ... añadir hasta 30 frases
];

// Función de selección por día del año (reproducible):
export function getDailyMessage(): string {
  const now = new Date();
  const start = new Date(now.getFullYear(), 0, 0);
  const diff = now.getTime() - start.getTime();
  const dayOfYear = Math.floor(diff / 86400000);
  const index = dayOfYear % DAILY_MESSAGES.length;
  return DAILY_MESSAGES[index];
}

// En app/(tabs)/stats.tsx — reemplazar:
// const message = MESSAGES[new Date().getDay()];   <- antes
const message = getDailyMessage();                  // <- ahora
```

# Orden de implementación recomendado

Seguir este orden minimiza el riesgo de romper funcionalidad existente al hacer los cambios:

1. **Migración de base de datos (Errores 1, 3 y 4)** — Primero ejecuta todos los cambios de schema: añadir repeat_notification_id a tasks, y crear la tabla daily_log. Así las funciones nuevas tienen donde escribir.
2. **Actualizar tipos TypeScript** — Añadir repeat_notification_id a la interfaz Task en db.ts y db.web.ts. Esto hará que TypeScript señale todos los sitios del código que hay que actualizar.
3. **Reescribir notifications.ts (Error 1)** — La nueva versión de scheduleTaskNotification es el cambio más central. Hazlo antes de tocar el store.
4. **Actualizar el store (Errores 1 y 2)** — Actualizar addTask, editTask, removeTask, toggleTask y añadir rescheduleAll.

5. **Añadir updateDailyLog en los puntos de escritura (Error 3)** — Conectar el log diario en todos los métodos del store que cambian el estado de tareas.
6. **Actualizar Stats para leer de daily_log (Errores 3 y 4)** — Reescribir calcStreak y el gráfico semanal para leer de la nueva tabla.
7. **Arreglar el toggle de notificaciones (Error 2)** — Añadir rescheduleAll al handler del switch en Settings.
8. **Extraer DayPicker (Deuda técnica 1)** — Crear el componente compartido y actualizar ambos modales. Es independiente y no rompe nada.
9. **Ampliar frases motivacionales (Deuda técnica 2)** — El cambio más pequeño. Dejarlo para el final.

---

Fin del documento — FocusOn Plan de corrección técnica v1.0